

```
"""
```

```
ECE 601 - Machine Learning for Engineers, Spring 2026
```

```
Course Project - Phase 3: Ablation Study
```

```
HAM10000 Skin Lesion Classifier
```

```
Experiments:
```

- 0 - Baseline (ResNet-18, weighted loss, augmentation, StepLR)
- 1 - No weighted loss (plain cross-entropy)
- 2 - No augmentation
- 3 - No LR scheduler
- 4 - + WeightedRandomSampler (oversample minority classes)
- 5 - + ResNet-50 backbone

```
Results saved to: ablation_results.csv
```

```
Plots saved to: ablation_plots/
```

```
Full log saved to: ablation_log.txt
```

```
Email sent to: ishrak021202@gmail.com when complete
```

```
"""
```

```
import os
```

```
import io
```

```
import random
```

```
import smtplib
```

```
import traceback
```

```
import numpy as np
```

```
import pandas as pd
```

```
from PIL import Image
```

```
from datetime import datetime
```

```
from email.mime.text import MIMEText
```

```
from email.mime.multipart import MIMEMultipart
```

```
from email.mime.base import MIMEBase
```

```
from email import encoders
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import f1_score, classification_report
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
```

```
from torchvision import transforms, models
```

```
import matplotlib
```

```
matplotlib.use("Agg") # non-interactive backend for running without display
```

```
import matplotlib.pyplot as plt
```

```

# -----
# 0. Config
# -----
SEED          = 42
DATA_DIR      = r"C:\pracona\601\HAM10000"
CSV_PATH      = os.path.join(DATA_DIR, "HAM10000_metadata.csv")
IMG_DIR_1     = os.path.join(DATA_DIR, "HAM10000_images_part_1")
IMG_DIR_2     = os.path.join(DATA_DIR, "HAM10000_images_part_2")
PLOT_DIR      = r"C:\pracona\601\ablation_plots"
LOG_PATH      = r"C:\pracona\601\ablation_log.txt"
CSV_OUT       = r"C:\pracona\601\ablation_results.csv"

IMG_SIZE      = 224
BATCH_SIZE    = 32
NUM_EPOCHS    = 20
LR            = 1e-4
NUM_WORKERS   = 0

EMAIL_SENDER  = "ishrak021202@gmail.com"
EMAIL_RECEIVER = "ishrak021202@gmail.com"
EMAIL_PASSWORD = "ouay nfjp ulel egmi"

os.makedirs(PLOT_DIR, exist_ok=True)

LABEL_MAP = {
    "mel": 0, "nv": 1, "bcc": 2,
    "akiec": 3, "bkl": 4, "df": 5, "vasc": 6,
}
CLASS_NAMES = list(LABEL_MAP.keys())

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# -----
# 1. Logging
# -----
log_buffer = []

def log(msg):
    timestamp = datetime.now().strftime("%H:%M:%S")
    line = f"[{timestamp}] {msg}"
    print(line)
    log_buffer.append(line)
    with open(LOG_PATH, "a") as f:
        f.write(line + "\n")

# -----

```

```

# 2. Reproducibility
# -----
def set_seed():
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

# -----
# 3. Dataset
# -----
def find_image_path(image_id):
    for img_dir in [IMG_DIR_1, IMG_DIR_2]:
        path = os.path.join(img_dir, image_id + ".jpg")
        if os.path.exists(path):
            return path
    return None

class SkinLesionDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.df = dataframe.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        image = Image.open(row["path"]).convert("RGB")
        label = int(row["label"])
        if self.transform:
            image = self.transform(image)
        return image, label

# -----
# 4. Transforms
# -----
train_transform_aug = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

```

```

train_transform_noaug = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

val_test_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

# -----
# 5. Training / evaluation helpers
# -----

def run_epoch(model, loader, criterion, optimizer=None, train=True):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.set_grad_enabled(train):
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)
            if train:
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()
            total_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += images.size(0)
    return total_loss / total, correct / total

def evaluate_test(model, loader):
    model.eval()
    all_preds, all_labels = [], []
    with torch.no_grad():
        for images, labels in loader:
            preds = model(images.to(DEVICE)).argmax(1).cpu().numpy()
            all_preds.extend(preds)
            all_labels.extend(labels.numpy())
    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)
    acc = (all_preds == all_labels).mean()
    macro_f1 = f1_score(all_labels, all_preds, average="macro")
    report = classification_report(all_labels, all_preds, target_names=CLASS_NAMES)
    return acc, macro_f1, report

```

```

def save_plot(train_losses, val_losses, train_accs, val_accs, name):
    epochs = range(1, len(train_losses) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].plot(epochs, train_losses, label="Train Loss")
    axes[0].plot(epochs, val_losses, label="Val Loss")
    axes[0].set_title(f"Loss - {name}")
    axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss"); axes[0].legend()
    axes[1].plot(epochs, train_accs, label="Train Acc")
    axes[1].plot(epochs, val_accs, label="Val Acc")
    axes[1].set_title(f"Accuracy - {name}")
    axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Accuracy"); axes[1].legend()
    plt.tight_layout()
    path = os.path.join(PLOT_DIR, f"{name.replace(' ', '_')}.png")
    plt.savefig(path, dpi=150)
    plt.close()
    log(f"Saved plot: {path}")
    return path

# -----
# 6. Single experiment runner
# -----

def run_experiment(name, train_df, val_df, test_df, df_full,
                  use_weighted_loss=True,
                  use_augmentation=True,
                  use_scheduler=True,
                  use_weighted_sampler=False,
                  backbone="resnet18"):

    log(f"\n{'='*60}")
    log(f"EXPERIMENT: {name}")
    log(f"{'='*60}")
    set_seed()

    # Transforms
    tr_transform = train_transform_aug if use_augmentation else train_transform_n

    # DataLoaders
    train_ds = SkinLesionDataset(train_df, tr_transform)
    val_ds = SkinLesionDataset(val_df, val_test_transform)
    test_ds = SkinLesionDataset(test_df, val_test_transform)

    if use_weighted_sampler:
        class_counts = df_full["label"].value_counts().sort_index().values
        sample_weights = [1.0 / class_counts[label] for label in train_df["label"]]
        sampler = WeightedRandomSampler(sample_weights, num_samples=len(sample_weights))

```

```

        train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, sampler=sample
else:
        train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,

val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_w
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False, num_w

# Model
if backbone == "resnet18":
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    model.fc = nn.Linear(model.fc.in_features, len(LABEL_MAP))
elif backbone == "resnet50":
    model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
    model.fc = nn.Linear(model.fc.in_features, len(LABEL_MAP))
model = model.to(DEVICE)

# Loss
if use_weighted_loss:
    class_counts = df_full["label"].value_counts().sort_index().values
    weights = torch.tensor(
        (1.0 / class_counts) / (1.0 / class_counts).sum() * len(LABEL_MAP),
        dtype=torch.float).to(DEVICE)
    criterion = nn.CrossEntropyLoss(weight=weights)
else:
    criterion = nn.CrossEntropyLoss()

# Optimizer & scheduler
optimizer = optim.Adam(model.parameters(), lr=LR)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1) if u

# Training
train_losses, val_losses, train_accs, val_accs = [], [], [], []
best_val_acc = 0.0
best_path = os.path.join(PLOT_DIR, f"{name.replace(' ', '_')}_best.pth")

for epoch in range(NUM_EPOCHS):
    tr_loss, tr_acc = run_epoch(model, train_loader, criterion, optimizer, tr
    va_loss, va_acc = run_epoch(model, val_loader, criterion, train=False)
    if scheduler:
        scheduler.step()
    train_losses.append(tr_loss); val_losses.append(va_loss)
    train_accs.append(tr_acc); val_accs.append(va_acc)
    log(f" Epoch [{epoch+1:02d}/{NUM_EPOCHS}] "
        f"Train Loss: {tr_loss:.4f} Train Acc: {tr_acc:.4f} | "
        f"Val Loss: {va_loss:.4f} Val Acc: {va_acc:.4f}")
    if va_acc > best_val_acc:
        best_val_acc = va_acc

```

```

        torch.save(model.state_dict(), best_path)

log(f" Best Val Acc: {best_val_acc:.4f}")

# Test evaluation
model.load_state_dict(torch.load(best_path, weights_only=True))
test_acc, macro_f1, report = evaluate_test(model, test_loader)
log(f" Test Acc: {test_acc:.4f} Macro F1: {macro_f1:.4f}")
log(f"\n{report}")

# Plot
plot_path = save_plot(train_losses, val_losses, train_accs, val_accs, name)

return {
    "Experiment": name,
    "Backbone": backbone,
    "Weighted Loss": use_weighted_loss,
    "Augmentation": use_augmentation,
    "LR Scheduler": use_scheduler,
    "Weighted Sampler": use_weighted_sampler,
    "Best Val Acc": round(best_val_acc, 4),
    "Test Accuracy": round(test_acc, 4),
    "Macro F1": round(macro_f1, 4),
    "Per-Class Report": report,
}

# -----
# 7. Email results
# -----

def send_email(results_df, plot_paths):
    log("\nSending results email...")
    msg = MIMEMultipart()
    msg["From"] = EMAIL_SENDER
    msg["To"] = EMAIL_RECEIVER
    msg["Subject"] = "ECE 601 Phase 3 – Ablation Study Complete"

    # Body: summary table
    body = "Phase 3 Ablation Study – Results Summary\n"
    body += "=" * 60 + "\n\n"
    summary_cols = ["Experiment", "Test Accuracy", "Macro F1"]
    body += results_df[summary_cols].to_string(index=False)
    body += "\n\n" + "=" * 60 + "\n\nFull Per-Class Reports:\n\n"
    for _, row in results_df.iterrows():
        body += f"--- {row['Experiment']} ---\n"
        body += row["Per-Class Report"] + "\n\n"
    body += "\nFull log attached.\n"
    msg.attach(MIMEText(body, "plain"))

```

```

# Attach log file
with open(LOG_PATH, "rb") as f:
    part = MIMEBase("application", "octet-stream")
    part.set_payload(f.read())
    encoders.encode_base64(part)
    part.add_header("Content-Disposition", f"attachment; filename=ablation_lo
msg.attach(part)

# Attach CSV
with open(CSV_OUT, "rb") as f:
    part = MIMEBase("application", "octet-stream")
    part.set_payload(f.read())
    encoders.encode_base64(part)
    part.add_header("Content-Disposition", f"attachment; filename=ablation_re
msg.attach(part)

# Attach plots
for path in plot_paths:
    with open(path, "rb") as f:
        part = MIMEBase("application", "octet-stream")
        part.set_payload(f.read())
        encoders.encode_base64(part)
        part.add_header("Content-Disposition", f"attachment; filename={os.pat
msg.attach(part)

try:
    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
        server.login(EMAIL_SENDER, EMAIL_PASSWORD)
        server.sendmail(EMAIL_SENDER, EMAIL_RECEIVER, msg.as_string())
    log("Email sent successfully!")
except Exception as e:
    log(f"Email failed: {e}")

# -----
# 8. Main
# -----
if __name__ == "__main__":
    log(f"Using device: {DEVICE}")
    log(f"Started: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

# Load data
def find_path(iid):
    for d in [IMG_DIR_1, IMG_DIR_2]:
        p = os.path.join(d, iid + ".jpg")
        if os.path.exists(p): return p
    return None

```



```

df = pd.read_csv(CSV_PATH)
df["label"] = df["dx"].map(LABEL_MAP)
df["path"] = df["image_id"].apply(find_path)
df = df.dropna(subset=["path"])
log(f"Dataset: {len(df)} images")

train_df, temp_df = train_test_split(df, test_size=0.30, stratify=df["label"])
val_df, test_df = train_test_split(temp_df, test_size=0.50, stratify=temp_d
log(f"Train: {len(train_df)} Val: {len(val_df)} Test: {len(test_df)}")

# Run all experiments
experiments = [
    dict(name="0 Baseline", use_weighted_loss=True, use_augment
    dict(name="1 No Weighted Loss", use_weighted_loss=False, use_augment
    dict(name="2 No Augmentation", use_weighted_loss=True, use_augment
    dict(name="3 No LR Scheduler", use_weighted_loss=True, use_augment
    dict(name="4 WeightedRandomSampler", use_weighted_loss=True, use_augment
    dict(name="5 ResNet-50", use_weighted_loss=True, use_augment
]

all_results = []
plot_paths = []

for exp in experiments:
    try:
        result = run_experiment(
            train_df=train_df, val_df=val_df, test_df=test_df, df_full=df, **
        plot_paths.append(os.path.join(PLOT_DIR, f"{exp['name']}.replace(' ',
        all_results.append(result)
    except Exception as e:
        log(f"ERROR in {exp['name']}: {e}")
        log(traceback.format_exc())

# Save CSV
results_df = pd.DataFrame(all_results)
results_df.to_csv(CSV_OUT, index=False)
log(f"\nResults saved to {CSV_OUT}")

# Print summary
log("\n" + "="*60)
log("FINAL SUMMARY")
log("="*60)
summary_cols = ["Experiment", "Test Accuracy", "Macro F1"]
log("\n" + results_df[summary_cols].to_string(index=False))
log(f"\nFinished: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

```

```
# Email results
send_email(results_df, plot_paths)
```